CSE 331: Computer Security Fundamentals

Spring 2026

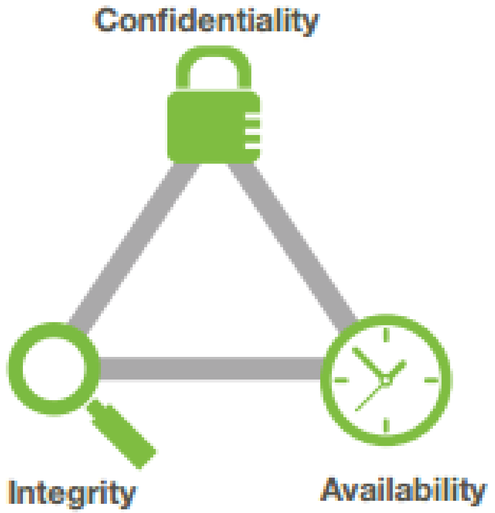
Access Control

Amir Rahmati
Xinrui Yu

Stony Brook Institute
at Anhui University
安徽大学纽约石溪学院

Protecting Data & Resources

- Confidentiality: Unauthorized Access
- Integrity: Improper modification
- Availability: no denial of service



Every access to a system and its resources must be controlled.

Access Control

Access control

- Need ways to *represent access control rights*
- Need ways to *grant specific rights*

- **What happens (in a computer security context) when you enter AHU via a specific entrance?**
- **Why can you get in here but can't get into other dorms?**

Access Control Matrix (Lampson, 1974)

- Abstract representation of a system's security state
  - Each cell defines the access rights for the given combination of subject and object
  - Empty cells mean that no access rights are granted
- *Objects:* set of protected entities
  - Files, directories, devices, resources, …
- *Subjects:* set of active objects
  - Users, groups, processes, systems, …
- *Rights:* set of possible operations
  - Read, write, execute, own, append, …

	File 1	File 2	File 3	Program 1
Ann	own read write	read write		execute
Bob	read		read write	
Carl		read		execute read

Access Control Matrix (Lampson, 1974)

	File 1	File 2	File 3	Program 1
Ann	own read write	read write		execute
Bob	read		read write	
Carl		read		execute read

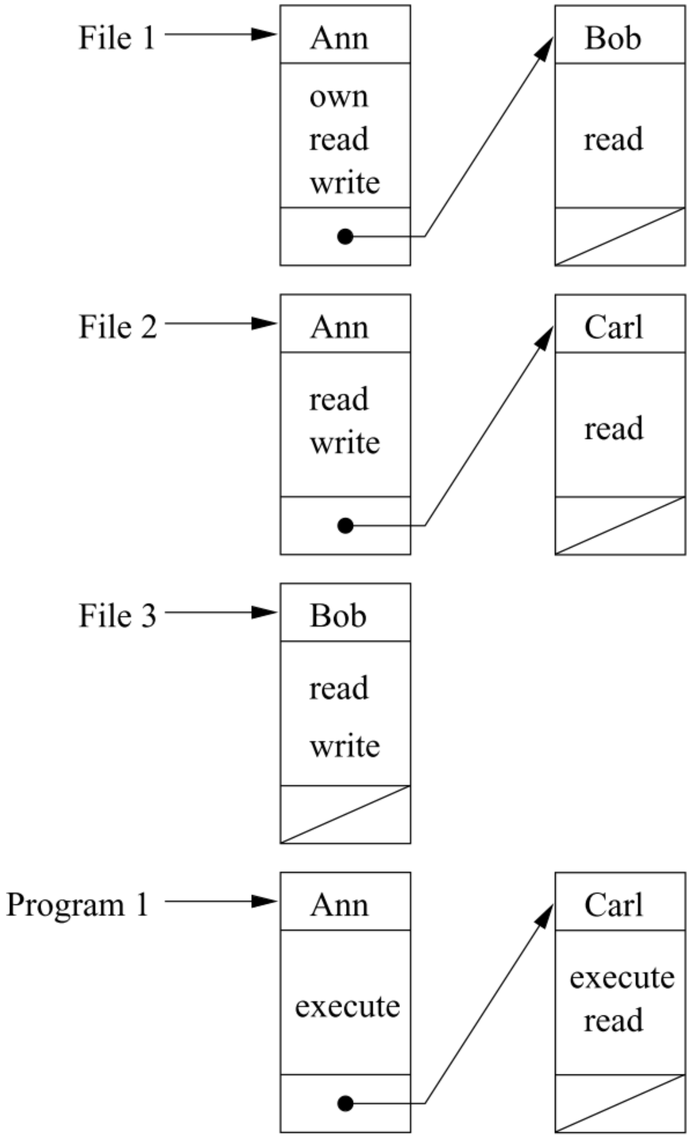
- Useful conceptual representation, but not practical
  - Size: number of subjects/objects will be too large
  - Efficiency: too many cells will be empty (no access) or the same (default permissions)
  - Management: addition/removal of subjects/objects requires non-trivial operations

Access Control Lists

- List of permissions attached to an object
  - Object-centered approach: enumeration of all subjects and their access rights for the given object
  - No entry in the ACL -> subject has no rights
- Facilitates the efficient implementation of default permissions
  - Wasteful to have separate entries for *many* different subjects that have the same rights over a given object
  - Better idea: define a “wildcard” to match any unnamed subject
  - Groups of subjects can also be represented in a similar way
- Tied to the object
  - Can be stored along with the object in the form of metadata

Access Control Lists

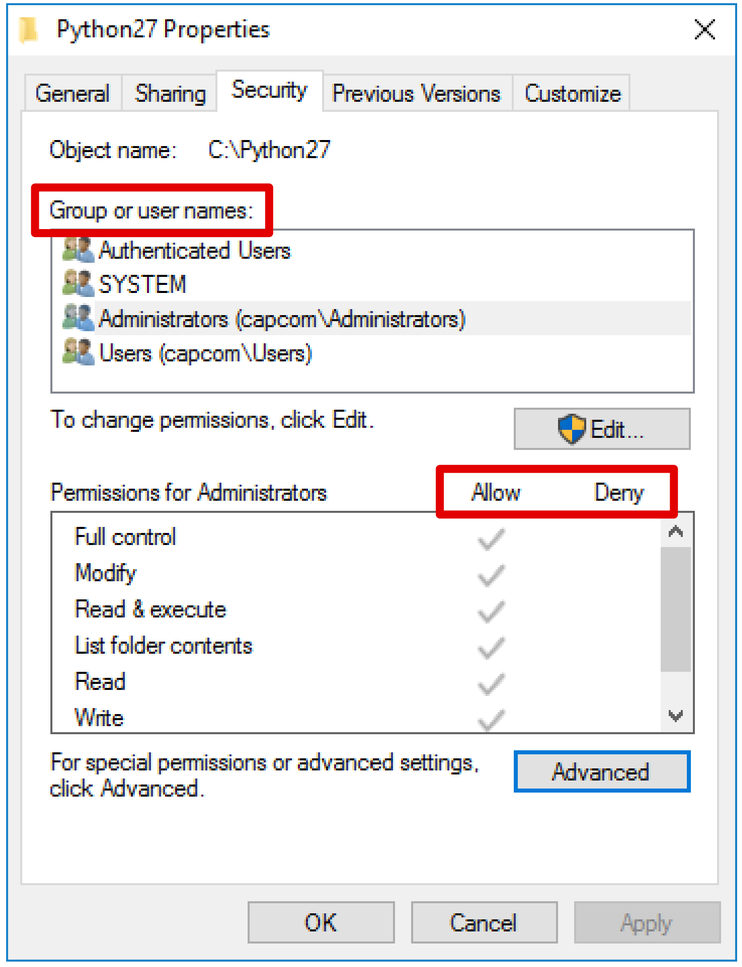
	File 1	File 2	File 3	Program 1
Ann	own read write	read write		execute
Bob	read		read write	
Carl		read		execute read

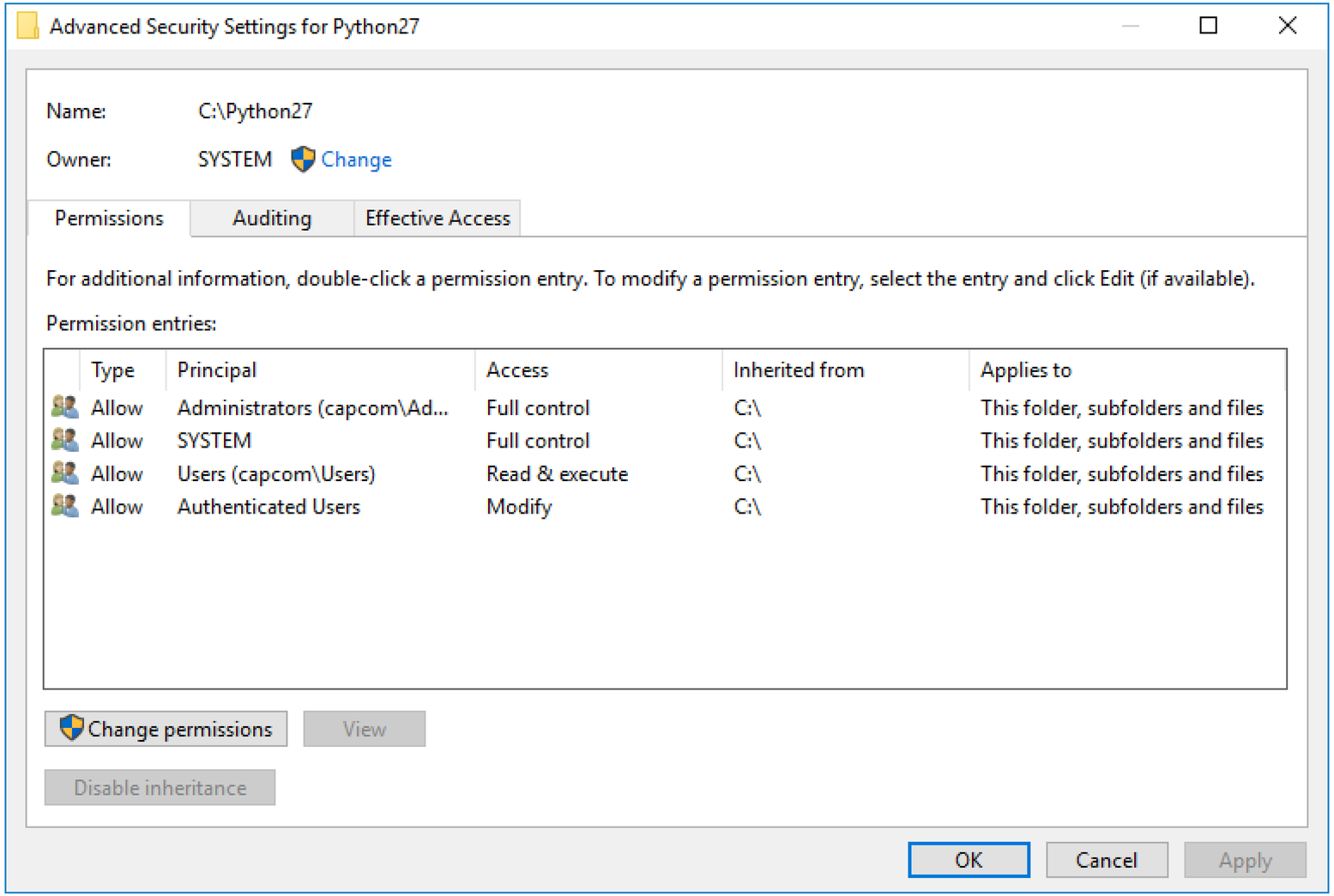


- Advantages: Dramatically smaller size (empty cells are collapsed)
- Disadvantages: No efficient way to enumerate all the rights of a given subject

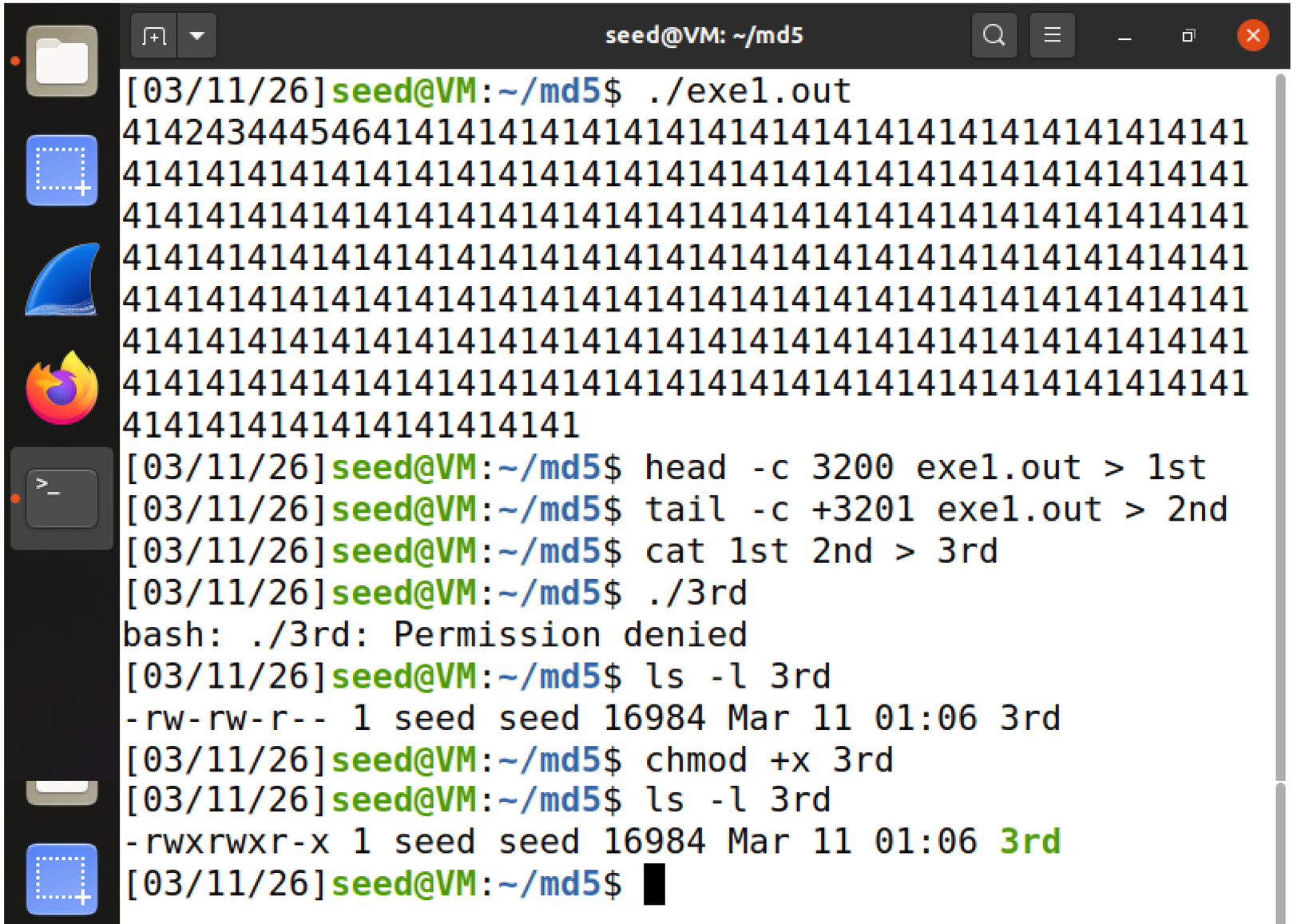
Example:
Campus Access Control System

Example: Windows File Permissions





Example: Linux Permissions

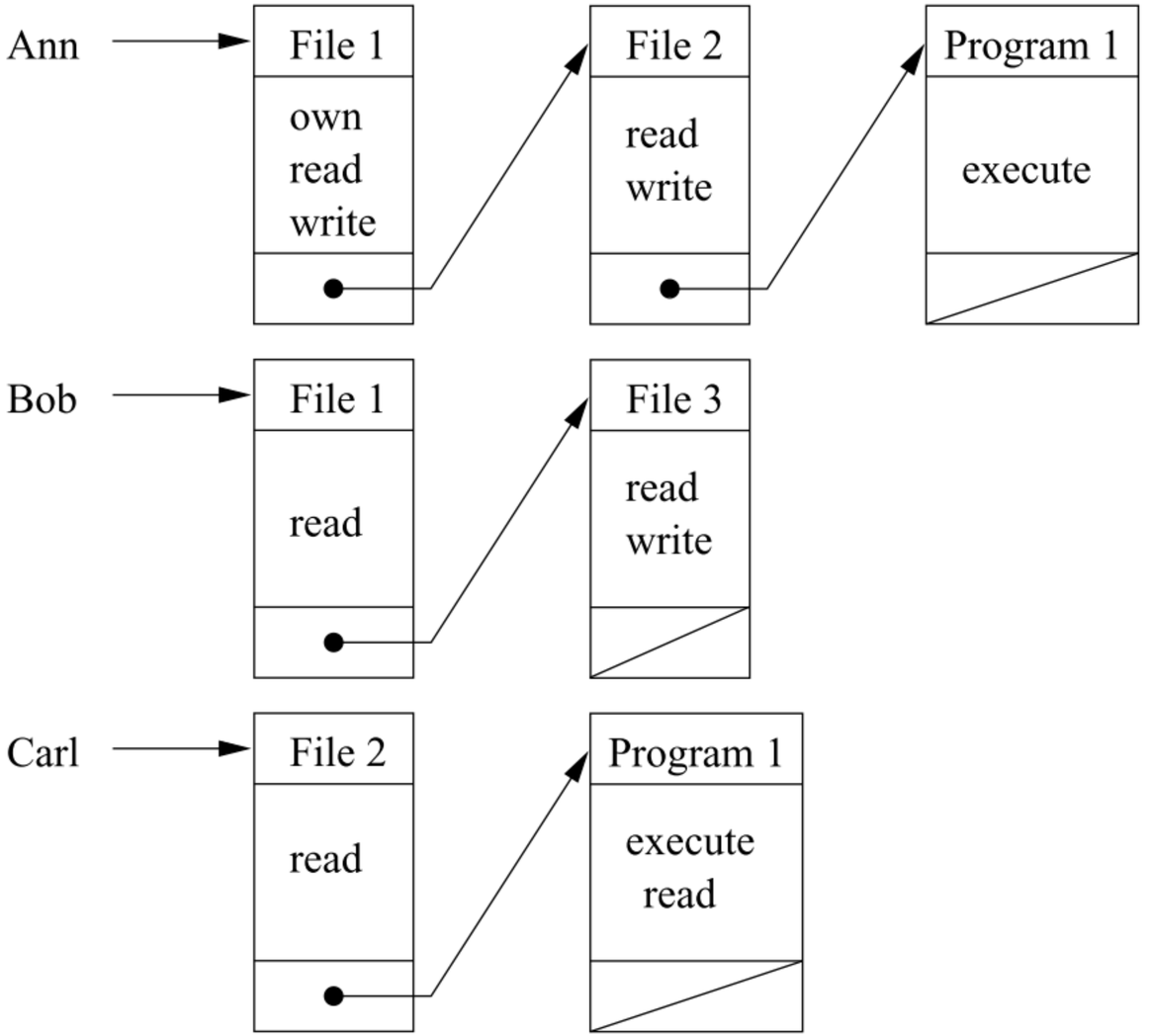


Capabilities

- List of access rights granted to a subject
  - Subject-centered approach: enumeration of all objects and the access rights a given subject has on them
- A capability can be a single token of authority
  - **Unforgeable:** must be validated
  - **Communicable:** can be passed on to other subjects or converted to less-privileged versions (in accordance to the enforced policy)
- Tied to the subject
  - Typically stored in a privileged data structure
  - The subject should not be able to forge access rights or change the object a capability is related to

Capabilities

- Advantages:
  - Dramatically smaller size (empty cells are collapsed)
  - Good for distributed systems. (No repeated authentication)
- Disadvantages:
  - Revocation can be challenging: No efficient way to enumerate all the rights on a given object
  - Can be susceptible to forgery



Example:
Hotel Manager’s Keycard

Question: ACL or Capabilities?

Metro

Capabilities

Railway

ACL

Access control

- Need ways to *represent access control rights*
  - Access control Matrix, ACLs, Capabilities, …

- Need ways to *grant specific rights*
  - DAC, MAC, RBAC, …

Access Control Policies

- Different approaches to granting access rights
  - Who is responsible? admin, user, owner, …
  - Based on what? identity, role, group, rule, …
- Main Types of Access Control
  - Discretionary
  - Mandatory
  - Role-based

Discretionary Access Control

- Subjects determine who has access to their objects
  - Key concept: *the owner* Determines the access rights of other subjects on that object
  - A subject with a certain access permission can pass it on to any other subject

- Commonly used in operating systems and cloud storages
  - E.g., Linux and Windows allow users to specify file and folder permissions based on ACLs
  - Google Drive, OneDrive, Baidu Netdisk
  - On existing files they can already access, or any new files they will create

Mandatory Access Control

- An administrator grants all access rights
  - Cannot be altered by subjects. Users cannot override decisions either accidentally or intentionally
- MAC-enabled systems allow policy administrators to implement strict organization-wide security policies
  - Multilevel security (MLS) and specialized military systems
- Getting traction in mainstream OSes as a means of minimizing abuse and preventing misconfigurations
  - Linux: SELinux, AppArmor
  - Windows: Mandatory Integrity Control
- Makes distinction between users and subjects

Example: The Bell-LaPadula Model

- Inspired by the military multilevel security
  - Used for document classification and personnel clearance
- Security levels
  - Each object is classified at one of the confidentiality levels
  - Each user obtains clearance at one of the confidentiality levels
- Example: MLS classification
  - Unclassified ⊴ Confidential ⊴ Secret ⊴ Top Secret
- Goal: protect the **confidentiality** of information
  - Prevent read access to objects at a confidentiality classification higher than the subject's clearance

Example: The Bell-LaPadula Model

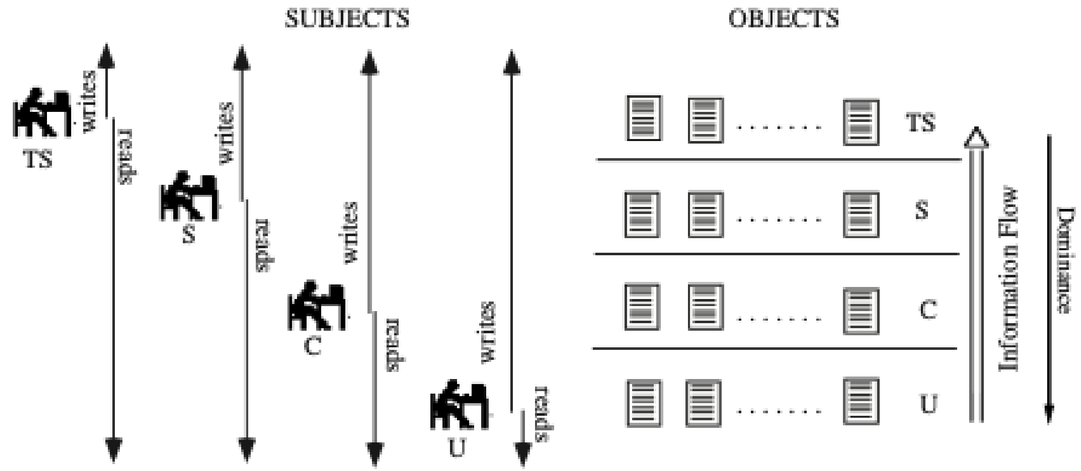
Access to objects is controlled by two rules:

No Read up

A subject cannot read an object of higher sensitivity

No Write down

A subject cannot write to an object of lower sensitivity



Example: Military Secret

Example: The Biba Model

- The Biba Model protects integrity
- Integrity levels
  - Each object is classified at one of the integrity levels
  - Each user obtains clearance at one of the integrity levels
- Example: MLS classification
  - Crucial ⊕ Important ⊕ Unknown
- Goal: protect the **integrity** of information
  - Prevent write access to objects at an integrity classification higher than the subject's clearance

Example: The Biba Model

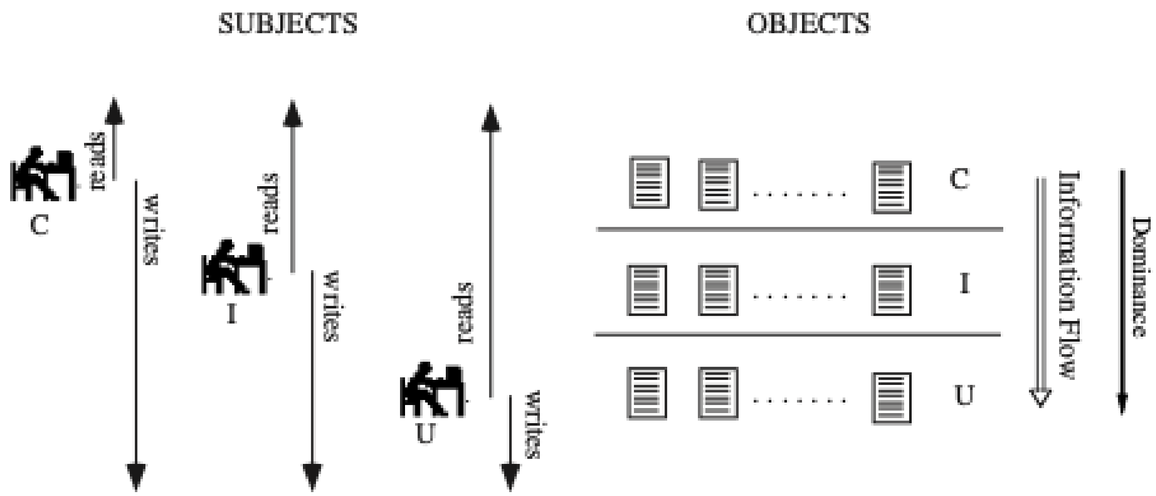
Access to objects is controlled by two rules:

No Read down

A subject cannot read to an object of lower integrity

No Write up

A subject cannot write an object of higher integrity



Example: State media, mass media, and social media

Downside of MAC

- Too rigid.
- Need to define exceptions
- Need to define methods to downgrade/upgrade level.

Role-based Access Control

- Access to an object is governed by the *role* of the subject within an organization
- Administrators define roles, specify access rights for each role, and assign subjects to roles
  - Much more convenient than managing subjects individually
- Example: roles for a computer science department
  - Student, faculty, administrative personnel, sysadmin, …
- Role hierarchies
  - More privileged roles may inherit the access rights of less privileged roles
  - Key difference from user groups

Rule-based Access Control

Allow or deny access to resources based on conditions other than the subject's identity

- Time of day, location, type of device, …
- Can be expressed in the form of complex Boolean logic rules